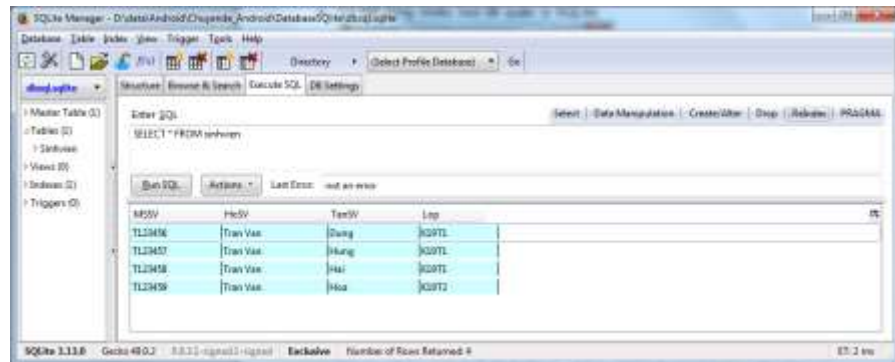


SQLite

1. SQLite là gì.

- SQLite là một Database mã nguồn mở dùng để lưu trữ dữ liệu nhiều và có cấu trúc được dùng khá rộng rãi ví dụ: FireFox dùng SQLite để lưu cấu hình, iphone cũng dùng SQLite cho database.... Bộ nhớ khi chạy rất nhỏ (khoảng 250Kbyte).
- SQLite hỗ trợ kiểu dữ liệu TEXT (tương tự kiểu String trong Java), INTEGER (tương tự kiểu long) và REAL (tương tự kiểu double). Các kiểu khác cần chuyển về các kiểu trên trước khi lưu xuống database. Bản thân SQLite không kiểm kiểu dữ liệu do đó có thể viết một integer vào một cột String và ngược lại.
- Có thể dùng nhiều tool để quản lý SQLite.
 - SQLite Manager (Addon Firefox)



- SQLite Studio



sqlitestudio-3.1.1.zip



2. SQLite và Android

- SQLite hỗ trợ mọi thiết bị Android. SQLite trong Android không cần phải cài đặt hay quản trị. Ta chỉ cần tạo câu lệnh SQL để tạo và cập nhật database sau đó database sẽ được quản lý bởi Android platform.
- Nếu chương trình tạo ra database thì database sẽ nằm trong đường dẫn:
- *data/data/<package name>/databases/<database name>*

3. Lớp SQLiteOpenHelper

- Để tạo và cập nhật các bảng ta cần phải tạo một lớp mới kế thừa từ lớp SQLiteOpenHelper. Trong lớp đó, hàm tạo gọi hàm super() và truyền vào tên và version của database.
- Trong class cần override 2 hàm là onCreate() và onUpgrade(). onCreate được gọi nếu database không tồn tại. onUpgrade() được gọi nếu version của database (trong mã lệnh) được thay đổi. Cả 2 hàm đều nhận một đối tượng SQLiteDatabase làm đối số.
- Cung cấp 2 hàm getReadableDatabase() và getWritableDatabase() để truy cập database theo chế độ là *đọc* hoặc *ghi*.
- Các bảng trong database nên dùng *id* làm khóa chính.

4. Lớp SQLiteDatabase

- Cung cấp các hàm mở, truy vấn, cập nhật và đóng database.

- Có hàm `execSQL()` dùng để thực thi câu lệnh SQL.
- Dùng đối tượng `ContentValues` để định nghĩa key/value. Key chỉ ra cột, value chỉ ra nội dung.
- Truy vấn dữ liệu bằng `rawQuery()` hoặc `query()`.

rawQuery():

```
Cursor cursor = getReadableDatabase().  
    rawQuery("select * from todo where _id = ?", new String[] { id });
```

query():

```
return database.query(DATABASE_TABLE,  
    new String[] { KEY_ROWID, KEY_CATEGORY, KEY_SUMMARY,  
    KEY_DESCRIPTION },  
    null, null, null, null, null);
```

Các đối số cho hàm query

Đối số	Mô tả
String dbName	Tên bảng
String[] columnNames	Danh sách cột trả về. null trả về tất cả các cột
String whereClause	Mệnh đề where, null lấy tất cả
String[] selectionArgs	Thay cho ?s trong “whereClause”
String[] groupBy	Mệnh đề groupby . null là không group
String[] having	Mệnh đề having. null là không lọc co group
String[] orderBy	Sắp xếp dữ liệu. null là không sắp xếp

Chú ý:

Mệnh đề where không có chữ “where” bên trong. Ví dụ: “_id=5 and ten=”

Trong mệnh đề where dùng các dấu “?” và dùng selectionArgs để đưa các biến vào thay thế cho các dấu “?”.

5. Lớp Cursor

- Một truy vấn sẽ trả về một đối tượng Cursor. Cursor trình bày tập kết quả của một truy vấn và trỏ đến hàng đầu tiên của tập kết quả (cách này giúp tiết kiệm bộ nhớ).
- Để duyệt các hàng kết quả ta dùng các hàm
 - moveToFirst() : nhảy đến hàng đầu tiên
 - moveToNext(): nhảy đến hàng tiếp theo
 - isAfterLast(): kiểm tra xem là hàng cuối cùng chưa.
 - getCount(): trả về tổng số phần tử.
- Cung cấp các hàm get*() để lấy kết quả trên các cột như getLong(), getString()....

Bài tập thực hành

Bài 1: Viết chương trình tạo và đọc database đơn giản.



Yêu cầu:

- Tên database: dbsví dụemo
- Tên table: sinhvien
- Cấu trúc table sinhvien như sau:

Tên trường	Kiểu dữ liệu	Mô tả
_id	Integer	Khóa chính (primary key) số nguyên tự tăng (AutoIncrement)

Hosv	String	Họ sinh viên
Tensv	String	Tên sinh viên
Lop	String	Lớp

- Đọc dữ liệu ra và hiển thị lên màn hình.

Hướng dẫn:

1. Tạo project mới. Trên layout của MainActivity chính thêm 1 TextView.
2. Tạo 1 class mới tên **Sinhvien** kế thừa từ lớp(Superclass) **SQLiteOpenHelper**. Khai báo một số hằng số. Override lên 2 hàm onCreate và onUpgrade. Tạo hàm tạo nhận 1 đối số.

Thư viện cần dùng:

```
import android.database.Cursor;
import android.content.Context;
import android.database.sqlite.SQLiteOpenHelper;
import android.database.sqlite.SQLiteDatabase;
import android.content.ContentValues;
```

```

public class Sinhvien extends SQLiteOpenHelper {

    private static final String DATABASE_NAME = "dbsvdemo";
    private static final int DATABASE_VERSION = 1;
    private static final String TABLE_NAME = "sinhvien";
    private static final String KEY_ID = "_id";
    private static final String KEY_FIRST = "hosv";
    private static final String KEY_LAST = "tensv";
    private static final String KEY_CLASS = "lop";

    public Sinhvien(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {

    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    }
}

```

3. Trong hàm onCreate viết thêm vào nội dung như sau:

```

@Override
public void onCreate(SQLiteDatabase db) {
    String sql = "create table " + TABLE_NAME +
        "(" +
        KEY_ID + " integer primary key autoincrement, " +
        KEY_FIRST + " text, " +
        KEY_LAST + " text, " +
        KEY_CLASS + " text" +
        ")";
    db.execSQL(sql);
}

```

4. Trong hàm onUpgrade viết thêm vào nội dung như sau:

```

@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    db.execSQL("drop table if exists " + TABLE_NAME);
    onCreate(db);
}

```

5. Xây dựng một hàm mới tên **taosv** nhận vào 3 đối số là ho, ten và lop và ghi dữ liệu vào database.

```

public void taosv(String ho, String ten, String lop) {
    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues noidung = new ContentValues();
    noidung.put(KEY_FIRST, ho);
    noidung.put(KEY_LAST, ten);
    noidung.put(KEY_CLASS, lop);

    String nullColumnHack = null; //chấp nhận field có giá trị null
    db.insert(TABLE_NAME, nullColumnHack, noidung);
}

```

6. Xây dựng một hàm mới tên **getAllSv** truy xuất tất cả dữ liệu trong bảng sinh viên và trả về một Cursor.

```

public Cursor getAllSv() {
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor con_tro = db.rawQuery("select * from " + TABLE_NAME, null);
    return con_tro;
}

```

7. Xây dựng thêm các hàm *xoasv*, *capnhatsv*, *timsv*

```

public boolean xoaSv(String _id){
    boolean done = false;
    SQLiteDatabase db = this.getWritableDatabase();
    String[] whereArgs = {_id};
    db.delete(TABLE_NAME, whereClause: KEY_ID + "= ?",whereArgs);
    return done;
}

public int capnhatsv(String _id,String ho,String ten,String lop)
{
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    ContentValues values = new ContentValues();
    values.put(KEY_FIRST,ho);
    values.put(KEY_LAST,ten);
    values.put(KEY_CLASS,lop);
    String[] whereArgs= {_id};

    int count =db.update(TABLE_NAME,contentValues, whereClause: this.KEY_ID+" = ?",whereArgs );
    return count;
}

public Cursor timsv(String searchText){
    SQLiteDatabase db = this.getReadableDatabase();
    String sql = "SELECT * FROM " + TABLE_NAME + " WHERE " + KEY_FIRST + " LIKE '%" + searchText + "%'";
    Cursor cursor = db.rawQuery(sql, selectionArgs: null);
    return cursor;
}

```

8. Quay về MainActivity chính. Trong hàm onCreate, tạo một đối tượng Sinhvien, thêm dữ liệu vào. Gọi hàm getAllSv để lấy tất cả dữ liệu và trả về một Cursor. Duyệt Cursor và lấy dữ liệu ra trình bày lên TextView.
Thư viện cần dùng:

```

import android.database.Cursor;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.TextView;

```



```

public class MainActivity extends AppCompatActivity {
    TextView tv;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        tv=(TextView) findViewById(R.id.txtview);

        Sinhvien qlsv=new Sinhvien(this);
        qlsv.taosv("Tran Van", "Dung", "K19T1");
        qlsv.taosv("Tran Van", "Hung", "K19T2");
        qlsv.taosv("Tran Van", "Hai", "K19T2");
        qlsv.taosv("Nguyen Van", "Toan", "K19T1");

        Cursor con_tro=qlsv.getAllSv();
        String chuoi="";
        con_tro.moveToFirst();
        do
        {
            chuoi+=con_tro.getString(0) + "\t\t\t";
            chuoi+=con_tro.getString(1) + "\t\t\t";
            chuoi+=con_tro.getString(2) + "\t\t\t";
            chuoi+=con_tro.getString(3) + "\n";
        }while(con_tro.moveToNext());

        tv.setText(chuoi);
    }
}

```

9. Kết thúc, chạy chương trình để thấy kết quả.

Bài 2: Viết chương trình tạo và đọc database theo dạng hướng đối tượng.

Giải thích:

Khi xây dựng ứng dụng có sử dụng database, nếu chương trình trở nên phức tạp ta nên xây dựng thêm những Class theo cấu trúc của bảng trong database. Trong lớp ta cài đặt các thuộc tính giống thuộc tính của bảng trong database và xây dựng thêm các hàm hỗ trợ. Khi đọc, ghi và xử lý dữ liệu, ta sẽ thông qua lớp này.

Ví dụ: trong bài trên, database chứa thông tin sinhvien, trong code ta nên xây dựng một lớp tên Sinhvien.

Bài toán khi viết mới nhìn có vẻ phức tạp vì có nhiều class hơn nhưng đối với bài toán lớn, cách viết như thế sẽ phát huy tác dụng giúp cho code dễ quản lý và gỡ lỗi hơn rất nhiều.

Yêu cầu:

- Tên database là quanlysinhvien
- Tên table là ds_sinhvien
- Cấu trúc ds_sinhvien như sau:

Tên trường	Kiểu dữ liệu	Mô tả
_id	Integer	Khóa chính (Primary key), kiểu số nguyên tự tăng (AutoIncrement)
hoten	String	Họ tên của sinhvien
Lop	String	Lớp của sinh viên

- Xây dựng class tên Sinhvien. Các thao tác trên bảng sinhvien phải thông qua class này.

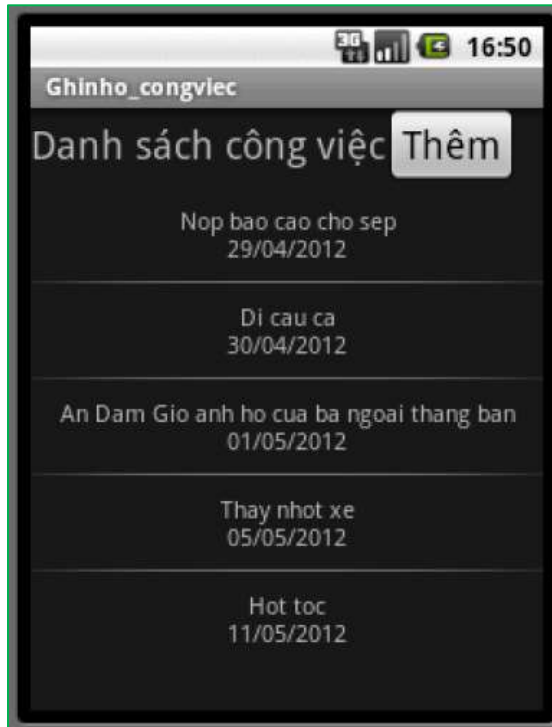
Hướng dẫn:

1. Tạo file Sinhvien.java và xây dựng class Sinhvien có:
 - 3 thuộc tính : **_id, hoten, lop**.
 - Xây dựng 3 hàm khởi tạo, 1 không có đối số truyền vào, 1 có 2 đối số truyền vào là **hoten** và **lop**, 1 có 3 đối số truyền vào là **_id, hoten** và **lop**.
 - Xây dựng 2 hàm get_id và set_id để gán và lấy thuộc tính _id.
 - Xây dựng 2 hàm là get_hoten và set_hoten để gán và lấy thuộc tính hoten.
 - Xây dựng 2 hàm là get_lop và set_lop để gán và lấy thuộc tính lop.
2. Tạo file quanlysinhvien.java và xây dựng Class quanlysinhvien kế thừa từ SQLiteOpenHelper gồm có:
 - Các thuộc tính DATABASE_VERSION, DATABASE_NAME, TEN_TABLE, KEY_ID, KEY_HOTEN, KEY_LOP.
 - Hàm tạo nhận 1 đối số và override 2 hàm là onCreate và onUpgrade.
 - Hàm thêm_Sinhvien giúp thêm 1 sinh viên mới.
 - Hàm get_Sinhvien nhận vào _id và trả về đối tượng sinhvien.
 - Hàm get_AllSinhvien trả về một List<Sinhvien>

- Hàm `soluong_Sinhvien` trả về số lượng sinh viên đang có.
- Hàm `delete_Sinhvien` nhận vào một đối tượng `Sinhvien` và xóa sinh viên đó.
- Hàm `update_Sinhvien` nhận vào một đối tượng `Sinhvien` và cập nhật sinh viên đó.

3. Xây dựng class chính gọi hàm `them_Sinhvien`, `update_sinhvien` và trả List `Sinhvien` để hiển thị.
4. Chạy chương trình và test dữ liệu.

Bài 3: Xây dựng ứng dụng GhiNho_CongViec đơn giản.



Yêu cầu:

- Tên database ghinho_congviec
- Tên table ds_congviec
- Cấu trúc table ds_congviec

Tên trường	Kiểu dữ liệu	Mô tả
_id	Integer	Khóa chính (primary key) kiểu số nguyên tự tăng (AutoIncrement)
Noidung	String	Nội dung công việc cần nhớ
Thoigian	String	Thời gian cần thực hiện công việc

- Viết chương trình theo kiểu hướng đối tượng.
- Khi nhấn nút “thêm” sẽ chuyển sang nhap_congviec_layout để thực hiện nhập mới một công việc vào database như hình:



Trên layout này khi nhấn vào nút “Lưu và trở về” thì dữ liệu được ghi vào database và layout kết thúc chuyển điều khiển về layout chính.

- Trên layout chính, khi muốn bỏ một công việc đang tồn tại hoặc bỏ tất cả công việc trong danh sách người dùng touch và giữ trong 2 giây trên một mục công việc để hiện lên Context Menu và cho phép chọn chức năng mong muốn.



Bài 4: Ghi nhớ công việc và mức độ quan trọng của công việc (phát triển từ bài 3).



Yêu cầu:

- Xây dựng theo kiểu hướng đối tượng. Phát triển từ bài 1 và thêm một số yêu cầu.
- Tên database ghinho_congviec2, Tên table ds_congviec2,
- Cấu trúc ds_congviec2

Tên trường	Kiểu dữ liệu	Mô tả
_id	Integer	Khóa chính (primary key) kiểu số nguyên tự tăng (AutoIncrement)
noidung	String	Nội dung công việc cần nhớ
thoigian	String	Thời gian cần thực hiện công việc
mucdo	String	Mức độ quan trọng của công việc.

- Thêm một trường mới vào database tên “mucdo” để qui định mức độ quan trọng của công việc, dữ liệu bên trong sẽ gồm “Đặc biệt”, “Bình thường” và “Không quan trọng”.
- Chức năng thêm công việc tương tự như bài 1.
- Khi hiển thị danh sách công việc. Mỗi công việc gồm 1 hình ảnh và text.
- Tùy theo mức độ của công việc sẽ hiển thị một trong 3 hình ảnh đại diện cho mức độ.
- Tùy theo mức độ của công việc mà nội dung cũng có màu khác nhau để dễ thấy.